APDIO Workshop: Optimization Software & Practice

Part 1: Modeling Basics

João Dionísio

March 27, 2026

Getting Started

1. Download the repository

Go to `https://github.com/Joao-Dionisio/apdio-workshop-2026`, click **Code** → **Download ZIP**, and extract it.

2. Install dependencies

```
pip install -r requirements.txt
```

This installs `pyscipopt` (Python interface to SCIP) and `matplotlib`.

3. Verify installation

```
python -c "from pyscipopt import Model; print('Ready!')"
```

Each exercise has a stub file and a test script. Edit the stub, then run the test: `python ex01_first_model/test_first_model.py`

Objectives

By the end of Part 1 you will be able to:

- Formulate problems in PySCIPOpt.
- Translate a mathematical model into Python code.
- Solve classical optimization problems:
  - Transportation, Portfolio Optimization, Set Cover,
  - Knapsack, Facility Location, Graph Coloring.
- Extract solutions and solver statistics.

Model Lifecycle

Create Model → **Add Variables** → **Add Constraints** → **Optimize** → **Query Solution**

```python
from pyscipopt import Model

model = Model("my_model")       # 1. Create
x = model.addVar(...)           # 2. Variables
model.addCons(...)              # 3. Constraints
model.optimize()                # 4. Optimize
print(model.getVal(x))          # 5. Query
```

Variables

`model.addVar(name, vtype, lb, ub, obj)`

vtype	Domain	Typical use
"C"	Continuous ($\mathbb{R}$)	Quantities, flows
"B"	Binary ($\{0, 1\}$)	Yes/no decisions
"I"	Integer ($\mathbb{Z}$)	Counts

```
# Binary variable with objective coefficient 3
x = model.addVar(name="x", vtype="B", obj=3)

# Continuous variable in [0, 10]
y = model.addVar(name="y", vtype="C", lb=0, ub=10)

# Integer variable, default bounds [0, +inf)
z = model.addVar(name="z", vtype="I", obj=-2)
```

Constraints

`model.addCons()`

```
# Single constraint
model.addCons(x + 2*y <= 10, name="capacity")

# Indexed family of constraints
for j in customers:
    model.addCons(
        sum(x[i,j] for i in suppliers) == demand[j],
        name=f"demand_{j}"
    )

model.addCons(sum(c[i]*x[i] for i in I) <= b)
```

Objective Function

`setObjective()`

```
model.setObjective(
    sum(cost[i,j] * x[i,j] for i in I for j in J),
    sense="maximize"
)
```

- Default sense is `"minimize"`

Exercise 1: First Model

Task

Build the following model:

$$
\begin{aligned}
\max \quad & 3x + 2y \\
\text{s.t.} \quad & x + y \leq 1 \\
& 2x + y \leq 2 \\
& x, y \in \{0, 1\}
\end{aligned}
$$

- File: `ex01_first_model/first_model.py`
- Test: `python ex01_first_model/test_first_model.py`

Solving and Querying

```
model.hideOutput()              # suppress solver log (optional)
model.optimize()                # solve the model

status = model.getStatus()      # "optimal", "infeasible", ...
obj    = model.getObjVal()      # objective value
val_x  = model.getVal(x)       # value of variable x
```

Common status strings:

Status	Meaning
`"optimal"`	Optimal solution found
`"infeasible"`	No feasible solution exists
`"unbounded"`	Objective is unbounded
`"timelimit"`	Time limit reached (may have incumbent)

Exercise 2: Solving

Task

Solve the model from Exercise 1 and extract:

- Optimal status, objective value, variable values.
- Solver statistics (number of nodes, LP iterations).

- File: `ex02_solving/solving.py`
- Test: `python ex02_solving/test_solving.py`

Solver Parameters

SCIP has over 2000 parameters to control the solving process.

Setting parameters in PySCIPOpt:

```
model.setParam("limits/time", 60)        # time limit (seconds)
model.setParam("limits/gap", 0.01)       # stop at 1% gap
model.setParam("limits/nodes", 1000)     # node limit

model.setParam("display/verblevel", 0)  # silence output
model.setParam("randomization/randomseedshift", 42)
```

Emphasis settings adjust groups of parameters at once:

```
model.setEmphasis("OPTIMALITY")  # focus on proving optimality
model.setEmphasis("FEASIBILITY") # focus on finding solutions
```

Reading models from file:

```
model = Model()
model.readProblem("instance.mps")  # or .lp
model.optimize()
```

Exercise 3: Solver Parameters

Task

Experiment with solver parameters:

- Set a time limit and observe the effect on a hard instance.
- Compare default vs. `FEASIBILITY` emphasis.
- Use `limits/gap` to accept near-optimal solutions.
- Load a model from an `.lp`/`.mps` file and tune parameters.

- File: `ex03_parameters/parameters.py`
- Test: `python ex03_parameters/test_parameters.py`

Exercise 3b: MIPLIB Instances

Task

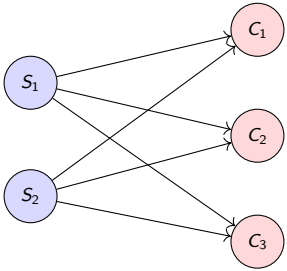
Load and solve real instances from MIPLIB, the standard benchmark library for mixed-integer programming. Collect: status, objective, time, nodes, gap.

```
model = Model()
model.readProblem(
    "miplib_data/p0033.mps.gz")
model.setParam("limits/time", 120)
model.optimize()
```

Pre-downloaded in `ex03_parameters/miplib_data/`:

Instance	Type	Vars	Conss
p0033	IP	33	16
flugpl	MIP	18	18
enigma	IP	100	21
misc03	MIP	160	96
stein27	IP	27	118
gen-ip054	IP	30	27

Transportation Problem



Sets: suppliers I, customers J
Parameters: supply s_i, demand d_j, cost c_{ij}
Variables: $x_{ij} \geq 0$ (flow from i to j)

$$
\begin{aligned}
\min \quad & \sum_{i \in I} \sum_{j \in J} c_{ij}\, x_{ij} & \\
\text{s.t.} \quad & \sum_{j \in J} x_{ij} \leq s_i & \forall\, i \in I \\
& \sum_{i \in I} x_{ij} \geq d_j & \forall\, j \in J \\
& x_{ij} \geq 0 &
\end{aligned}
$$

Data in Python — Dictionaries and Loops

```python
# Data as dictionaries
supply   = {"S1": 100, "S2": 150}
demand   = {"C1": 80, "C2": 90, "C3": 70}
cost     = {("S1","C1"): 4, ("S1","C2"): 6, ("S1","C3"): 9,
            ("S2","C1"): 5, ("S2","C2"): 3, ("S2","C3"): 7}

# Variables via nested comprehension
x = {(i,j): model.addVar(name=f"x_{i}_{j}", vtype="C")
     for i in supply for j in demand}

# Constraints via loops
for i in supply:
    model.addCons(
        quicksum(x[i,j] for j in demand) <= supply[i])
for j in demand:
    model.addCons(
        quicksum(x[i,j] for i in supply) >= demand[j])
```

Exercise 4: Transportation

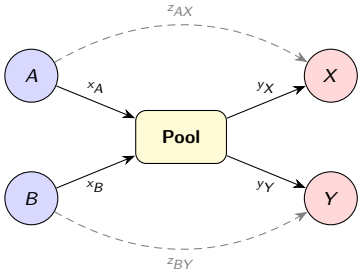
Task

Implement the transportation problem:

- Read data from provided dictionaries.
- Build variables, supply/demand constraints, minimize cost.
- Print optimal shipment plan.

- File: `ex03_transportation/transportation.py`
- Test: `python ex03_transportation/test_transportation.py`

Nonlinear Blending (Pooling Problem)



dashed = direct bypass

Blend raw materials through a mixing pool to make products with quality specifications.

Parameters: cost c_s, quality q_s per source; revenue r_p, max quality $\bar{q}_p$ per product

Variables: x_s (source to pool), y_p (pool to product), z_{sp} (direct bypass), λ (pool quality)

$$
\begin{aligned}
\max \quad & \textstyle\sum_p r_p\, d_p - \sum_s c_s \left(x_s + \sum_p z_{sp} \right) & \\
\text{s.t.} \quad & \textstyle\sum_s x_s = \sum_p y_p & \text{(balance)} \\
& \lambda \textstyle\sum_s x_s = \sum_s q_s\, x_s & \text{(pool qual.)} \\
& \lambda\, y_p + \textstyle\sum_s q_s\, z_{sp} \le \bar{q}_p\, d_p & \forall\, p
\end{aligned}
$$

where $d_p = y_p + \sum_s z_{sp}$.

Note: The terms $\lambda\, x_s$ and $\lambda\, y_p$ are **bilinear**. This is a nonconvex NLP that SCIP can solve globally.

Exercise 5: Nonlinear Blending

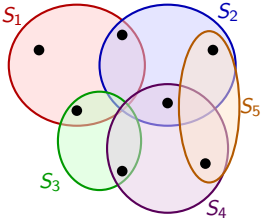
Task

Implement the pooling problem:

- Flow variables for pool and direct bypass paths.
- Pool quality variable λ with **bilinear constraints**.
- Maximize profit (linear objective).

- File: `ex04_blending/blending.py`
- Test: `python ex04_blending/test_blending.py`

From LP to IP: Set Cover



Select cheapest combination of sets to cover all elements.

Given: Universe $U = \{1, \ldots, n\}$, subsets $S_1, \ldots, S_m \subseteq U$, costs c_j.
Goal: Select minimum-cost subsets that cover every element.

Variables: $x_j \in \{0, 1\}$ (select subset j?)

$$\begin{aligned} \min \quad & \sum_{j=1}^{m} c_j x_j \\ \text{s.t.} \quad & \sum_{j:\, i \in S_j} x_j \geq 1 && \forall\, i \in U \\ & x_j \in \{0, 1\} && \forall\, j \end{aligned}$$

- This is a **pure integer program** (all variables binary).
- LP relaxation gives a lower bound; SCIP uses branch-and-bound.

Exercise 6: Set Cover

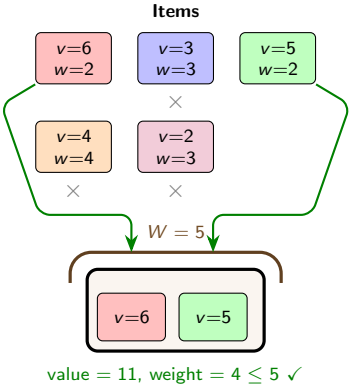
Task

Implement the set cover problem:

- Define binary variables for each subset.
- Add covering constraints for each element.
- Minimize total cost.

- File: `ex05_set_cover/set_cover.py`
- Test: `python ex05_set_cover/test_set_cover.py`

Knapsack Problem



Classic 0–1 Knapsack: select items to maximize value without exceeding capacity.

Applications: budgeting, cargo loading, resource allocation.

Variables: $x_j \in \{0, 1\}$ (take item j?)

$$\begin{aligned} \max \quad & \sum_{j=1}^{n} v_j \, x_j \\ \text{s.t.} \quad & \sum_{j=1}^{n} w_j \, x_j \leq W \\ & x_j \in \{0, 1\} \qquad \forall j \end{aligned}$$

where v_j = value, w_j = weight, W = capacity.

Exercise 7: Knapsack

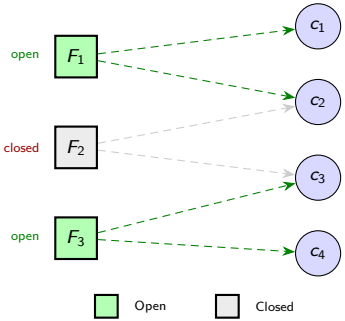
Task

Implement the 0–1 knapsack problem:

- Create binary variables for each item.
- Add the capacity constraint.
- Maximize total value and report selected items.

- File: `ex06_knapsack/knapsack.py`
- Test: `python ex06_knapsack/test_knapsack.py`

Facility Location



Decide which facilities to open and how to assign customers.

Sets: facilities F, customers C

Parameters: fixed cost f_i, assignment cost t_{ij}, demand d_j, capacity q_i

Variables: $y_i \in \{0,1\}$ (open i?), $x_{ij} \geq 0$ (fraction served)

$$
\begin{aligned}
\min \quad & \sum_{i \in F} f_i\, y_i + \sum_{i \in F} \sum_{j \in C} t_{ij}\, x_{ij} & \\
\text{s.t.} \quad & \sum_{i \in F} x_{ij} = 1 & \forall\, j \in C \\
& \sum_{j \in C} d_j\, x_{ij} \leq q_i\, y_i & \forall\, i \in F \\
& x_{ij} \geq 0,\ y_i \in \{0,1\} &
\end{aligned}
$$

- The constraint $x_{ij} \leq y_i$ **links** assignment to opening.

Exercise 8: Facility Location

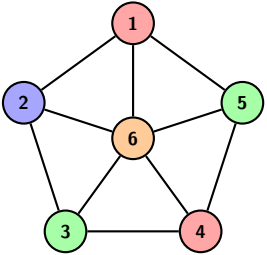
Task

Implement the uncapacitated facility location problem:

- Binary variables for facility opening decisions.
- Continuous variables for customer assignments.
- Linking and demand constraints.

- File: `ex07_facility_location/facility_location.py`
- Test: `python ex07_facility_location/test_facility_location.py`

Graph Coloring



Given: undirected graph $G = (V, E)$, set of colors $K = \{1, \ldots, k\}$.
Goal: minimize the number of colors used such that no two adjacent vertices share a color.

Variables: $x_{v,c} \in \{0, 1\}$ (vertex v gets color c?), $w_c \in \{0, 1\}$ (color c used?)

$$
\begin{aligned}
\min \quad & \sum_{c \in K} w_c \\
\text{s.t.} \quad & \sum_{c \in K} x_{v,c} = 1 && \forall\, v \in V \\
& x_{u,c} + x_{v,c} \leq w_c && \forall\, (u, v) \in E,\ c \in K \\
& x_{v,c},\ w_c \in \{0, 1\}
\end{aligned}
$$

Symmetry: permuting colors $\Rightarrow$ equivalent solution.
Break: $w_1 \geq w_2 \geq \cdots \geq w_k$.

Exercise 9: Graph Coloring

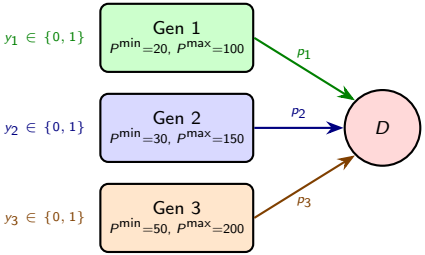
Task

Implement the graph coloring formulation:

- Assignment and conflict constraints.
- Minimize number of colors.
- Add symmetry-breaking constraints.

- File: `ex08_graph_coloring/graph_coloring.py`
- Test: `python ex08_graph_coloring/test_graph_coloring.py`

Generator Scheduling



Given: generators with on/off decisions, min/max output, and costs. **Goal:** meet demand D at minimum cost.

Key constraint: if on (y_i=1), output satisfies $P_i^{\min} \leq p_i \leq P_i^{\max}$. If off ($y_i$=0), $p_i = 0$.

$$\begin{aligned} \min \quad & \sum_i (f_i\, y_i + c_i\, p_i) \\ \text{s.t.} \quad & \sum_i p_i \geq D \\ & y_i = 0 \;\Rightarrow\; p_i = 0 \\ & y_i = 1 \;\Rightarrow\; P_i^{\min} \leq p_i \leq P_i^{\max} \end{aligned}$$

How do we model the conditional logic?

Indicator Constraints

Big-M constraints model logical implications using a large constant M. They work, but choosing M and the resulting LP relaxation can be tricky.

Indicator constraints link a binary variable directly to a linear constraint, no M needed:

$$y = 1 \;\Rightarrow\; \text{constraint holds}$$

Big-M:

$$p_i \leq P_i^{\max} \cdot y_i$$
$$p_i \geq P_i^{\min} \cdot y_i$$

Indicator:

```
addConsIndicator(p[i] >= P_min[i], y[i])
```

- SCIP handles the logic internally (branching, propagation).
- Often tighter LP relaxation and faster solving.

Exercise 10: Indicator Constraints

Task

Model a generator scheduling problem two ways:

- Big-M formulation (standard linearization).
- Indicator constraints (`model.addConsIndicator`).

Verify that both yield the same optimal solution.

- File: `ex09_indicators/indicators.py`
- Test: `python ex09_indicators/test_indicators.py`

Exercise 12: Benchmarking Formulations

Goal: measure how formulation choices affect solver performance.

Task

Generate generator scheduling instances of increasing size ($n = 10, 20, 50, 100, 200$). Solve with both big-M and indicator formulations. Compare:

- Solve time, number of B&B nodes, LP gap

Questions to answer:

- Does one formulation consistently solve faster?
- How does the gap between them change as n grows?
- Which formulation explores fewer B&B nodes?

- File: `ex10_benchmarking/benchmarking.py`
- Run: `python ex10_benchmarking/benchmarking.py`

Part 1 — Wrap-up

What we covered:

- The PySCIPOpt modeling lifecycle: variables, constraints, objective.
- Solving, parameters, and reading models from file.
- Classical LP/IP/MIP formulations:
  - Transportation (LP)
  - Nonlinear Blending (NLP)
  - Set Cover (IP)
  - Knapsack (IP)
  - Facility Location (MIP)
  - Graph Coloring (IP)
- Indicator constraints as an alternative to big-M.
- Benchmarking formulations on random and MIPLIB instances.

Up next

- Part 2: Row generation (TSP, cutting planes)
- Part 3: Column generation & branch-and-price (bin packing)